

Laboratorio de Computación Concurrente

Práctica 1

Hilos en Java

1. ¿Cómo creamos hilos en Java?

Hilos en Java

1. ¿Cómo creamos hilos en Java?

Opción 1:

Extendiendo la clase `Thread`

Redefinimos el método `run()`

Lo ejecutamos con `start()`

```
J SadThread.java > ...
1  public class SadThread extends Thread {
2
3      public void run() {
4          System.out.println(x:"En el run :( ");
5      }
6
7  }
8
9  }
```

```
J HappyThread.java > Language Support for Java(TM) by Red Hat > HappyThread
1  public class HappyThread extends Thread{
2
3      public void run() {
4          System.out.println(x:"Estamos en el run :) ");
5      }
6
7  }
```

```

1  }
J TestThread.java > Language Support for Java(TM) by Red Hat > TestThread
1  public class TestThread {
2
3      Run | Debug | Run main | Debug main
4      public static void main(String[] args) {
5
6          Thread happyT = new HappyThread();
7          Thread sadT = new SadThread();
8
9          happyT.start();
10         sadT.start();
11
12     }
13
14 }
15
16 }
```


Hilos en Java

1. ¿Cómo creamos hilos en Java?

Opción 2:

Implementando la interfaz `Runnable`

Redefinimos el método `run()`

Lo ejecutamos con `start()`

```
J PokerThread.java > Language Support for Java(TM) by Red Hat > PokerThr
1  public class PokerThread implements Runnable {
2
3      public void run() {
4          System.out.println(x:"Run Poker :| ");
5      }
6  }
7
```

```
J TestThread.java > ...
1  public class TestThread {
2
3      Run | Debug | Run main | Debug main
4      public static void main(String[] args) {
5
6          Thread happyT = new HappyThread();
7          Thread sadT = new SadThread();
8          PokerThread pokerT = new PokerThread();
9
10         happyT.start();
11         sadT.start();
12         pokerT.start();
13     }
14 }
15
J2
```


Hilos en Java

1. ¿Cómo creamos hilos en Java?

Opción 2:

Implementando la interfaz `Runnable`

Redefinimos el método `run()`

Lo ejecutamos con `start()`

Nota: El constructor de `Thread` permite crear hilos a partir de objetos que implementan la interfaz `Runnable`

Constructor Summary	
Constructors	
Constructor	Description
<code>Thread()</code>	Allocates a new Thread object.
<code>Thread(Runnable target)</code>	Allocates a new Thread object.
<code>Thread(Runnable target, String name)</code>	Allocates a new Thread object.
<code>Thread(String name)</code>	Allocates a new Thread object.
<code>Thread(ThreadGroup group, Runnable target)</code>	Allocates a new Thread object.
<code>Thread(ThreadGroup group, Runnable target, String name)</code>	Allocates a new Thread object so tha
<code>Thread(ThreadGroup group, Runnable target, String name, long stackSize)</code>	Allocates a new Thread object so tha the specified <i>stack size</i> .
<code>Thread(ThreadGroup group, String name)</code>	Allocates a new Thread object.

```
1 public class TestThread {
2
3     public static void main(String[] args) {
4
5         Thread happyT = new HappyThread();
6         Thread sadT = new SadThread();
7         PokerThread pokerT = new PokerThread();
8         Thread t1 = new Thread(pokerT);
9         new Thread(new Runnable() {
10
11             public void run() {
12                 System.out.println(x:"Hilo :/ ");
13             }
14         }).start();
15
16         happyT.start();
17         sadT.start();
18         pokerT.run();
19         t1.start();
20
21     }
22 }
```


Hilos en Java

Usando **clases anónimas**, podemos hacer algo del siguiente estilo.

1. ¿Cómo creamos hilos en Java?

Opción 2:

Implementando la interfaz **Runnable**

Redefinimos el método **run()**

Lo ejecutamos con **start()**

Nota: El constructor de **Thread** permite crear hilos a partir de objetos que implementan la interfaz **Runnable**

```
1  public class TestThread {
3      public static void main(String[] args) {
5          Thread happyT = new HappyThread();
6          Thread sadT = new SadThread();
7          PokerThread pokerT = new PokerThread();
8          Thread t1 = new Thread(pokerT);
9          new Thread(new Runnable() {
10
11              public void run() {
12                  System.out.println(x:"Hilo :/ ");
13              }
14          }).start();
15
16          happyT.start();
17          sadT.start();
18          pokerT.start();
19          t1.start();
20
21      }
22  }
```


Hilos en Java

2. Memoria Local vs Memoria Compartida

¿Qué se imprime?

```
public class ExampleMultipleExtends {  
  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        ThreadExtend1 threadA = new ThreadExtend1();  
        ThreadExtend1 threadB = new ThreadExtend1();  
        ThreadExtend1 threadC = new ThreadExtend1();  
        threadA.start();  
        threadB.start();  
        threadC.start();  
    }  
}  
  
class ThreadExtend1 extends Thread{  
    int count=0;  
    int increment() {  
        count++;  
        return count;  
    }  
    public void run() {  
        String threadName = Thread.currentThread().getName();//Obtenemos el nombre del hilo  
        System.out.println("Running Thread: " + threadName + " increment to: " + increment());  
    }  
}
```


Hilos en Java

2. Memoria Local vs Memoria Compartida

¿Qué se imprime?

Análisis: Se crean tres objetos cada uno a partir de una **nueva** instancia de `ThreadExtend1()` así, cada uno tiene su propia variable `count`, por lo que se obtiene algo como `1, 1, 1`

```
public class ExampleMultipleExtends {  
  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        ThreadExtend1 threadA = new ThreadExtend1();  
        ThreadExtend1 threadB = new ThreadExtend1();  
        ThreadExtend1 threadC = new ThreadExtend1();  
        threadA.start();  
        threadB.start();  
        threadC.start();  
    }  
}  
  
class ThreadExtend1 extends Thread{  
    int count=0;  
    int increment() {  
        count++;  
        return count;  
    }  
    public void run() {  
        String threadName = Thread.currentThread().getName(); //Obtenemos el nombre del hilo  
        System.out.println("Running Thread: " + threadName + " increment to: " + increment());  
    }  
}
```


Hilos en Java

2. Memoria Local vs Memoria Compartida

Si queremos un comportamiento de contador, ¿Qué podemos hacer?

Tenemos dos opciones;

- Podemos definir `count` como `static`.
- Usar un único objeto compartido.

```
public class ExampleMultipleExtends {  
  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        ThreadExtend1 threadA = new ThreadExtend1 ();  
        ThreadExtend1 threadB = new ThreadExtend1 ();  
        ThreadExtend1 threadC = new ThreadExtend1 ();  
        threadA.start ();  
        threadB.start ();  
        threadC.start ();  
    }  
}  
  
class ThreadExtend1 extends Thread{  
    int count=0;  
    int increment() {  
        count++;  
        return count;  
    }  
    public void run() {  
        String threadName = Thread.currentThread().getName(); //Obtenemos el nombre del hilo  
        System.out.println("Running Thread: " + threadName + " increment to: " + increment());  
    }  
}
```


Hilos en Java

2. Memoria Local vs Memoria Compartida

```
7 public class ExampleMultipleRunnable {
    Run | Debug | Run main | Debug main
8     public static void main(String[] args) {
9         ThreadRunnable1 myRunnable = new ThreadRunnable1();
10        Thread threadA = new Thread(myRunnable);
11        Thread threadB = new Thread(myRunnable);
12        Thread threadC = new Thread(myRunnable);
13        threadA.start();
14        threadB.start();
15        threadC.start();
16    }
17
18    class ThreadRunnable1 implements Runnable{
19        int count=0;
20        int increment() {
21            return count++;
22        }
23        int getCount() {
24            return count;
25        }
26        @Override
27        public void run() {
28            String threadName = Thread.currentThread().getName();//Obtenemos el nombre del hilo
29            System.out.println("Running Thread Runnable " + threadName + " increment to: " + increment());
30        }
31    }
32 }
```

¿Qué se imprime?

Hilos en Java

2. Memoria Local vs Memoria Compartida

```
7 public class ExampleMultipleRunnable {
8     public static void main(String[] args) {
9         ThreadRunnable1 myRunnable = new ThreadRunnable1();
10        Thread threadA = new Thread(myRunnable);
11        Thread threadB = new Thread(myRunnable);
12        Thread threadC = new Thread(myRunnable);
13        threadA.start();
14        threadB.start();
15        threadC.start();
16    }
17
18    class ThreadRunnable1 implements Runnable{
19        int count=0;
20        int increment() {
21            return count++;
22        }
23        int getCount() {
24            return count;
25        }
26        @Override
27        public void run() {
28            String threadName = Thread.currentThread().getName();//Obtenemos el nombre del hilo
29            System.out.println("Running Thread Runnable " + threadName + " increment to: " + increment());
30        }
31    }
```

¿Qué se imprime?

Nota 1. El orden en que aparecen los mensajes en la consola no necesariamente refleja el orden real de ejecución de los hilos

```
Running Thread Runnable Thread-0 increment to: 0
Running Thread Runnable Thread-2 increment to: 2
Running Thread Runnable Thread-1 increment to: 1
```


Hilos en Java

2. Memoria Local vs Memoria Compartida

```
7 public class ExampleMultipleRunnable {
8     public static void main(String[] args) {
9         ThreadRunnable1 myRunnable = new ThreadRunnable1();
10        Thread threadA = new Thread(myRunnable);
11        Thread threadB = new Thread(myRunnable);
12        Thread threadC = new Thread(myRunnable);
13        threadA.start();
14        threadB.start();
15        threadC.start();
16    }
17
18    class ThreadRunnable1 implements Runnable{
19        int count=0;
20        int increment() {
21            return count++;
22        }
23        int getCount() {
24            return count;
25        }
26        @Override
27        public void run() {
28            String threadName = Thread.currentThread().getName();//Obtenemos el nombre del hilo
29            System.out.println("Running Thread Runnable " + threadName + " increment to: " + increment());
30        }
31    }
32 }
```

¿Qué se imprime?

Nota 1. El orden en que aparecen los mensajes en la consola no necesariamente refleja el orden real de ejecución de los hilos

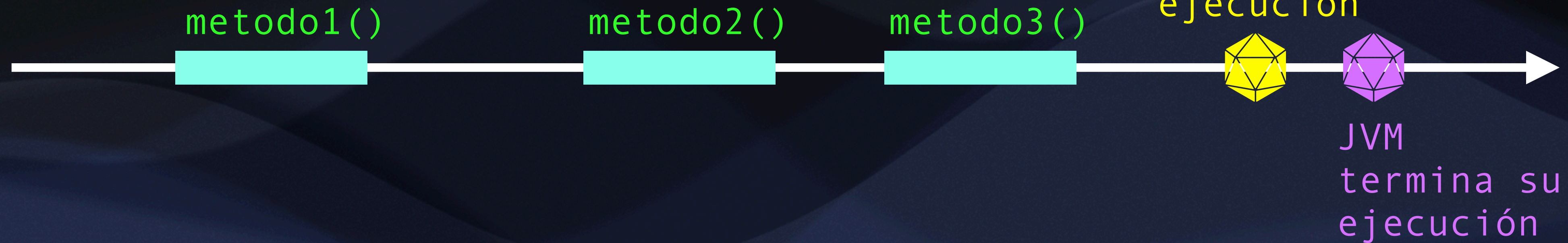
Nota 2. Aunque aparentemente ya hay un comportamiento correcto, eventualmente podremos encontrar ejecuciones en las que dos hilos incrementen lo mismo.

```
Running Thread Runnable Thread-1 increment to: 1
Running Thread Runnable Thread-0 increment to: 0
Running Thread Runnable Thread-2 increment to: 1
```


Hilos en Java

3. Control de vida de los Hilos

Supongamos que ejecutamos este programa, su ejecución se vería mas o menos de la siguiente manera



```
1 public class SeqExe {  
2  
3     public static void metodo1() {}  
4     public static void metodo2() {}  
5     public static void metodo3() {}  
6  
7     Run | Debug | Run main | Debug main  
8     public static void main(String[] args) {  
9         // Ejecución secuencial en el hilo principal  
10        metodo1();  
11        metodo2();  
12        metodo3();  
13  
14        System.out.println(x:"Todos los métodos se ejecutaron.");  
15        System.out.println(x:"El hilo main terminó, la JVM se apaga.");  
16    }  
17 }  
18  
19 }  
20 }  
21 }  
22 }  
23 }  
24 }  
25 }  
26 }  
27 }  
28 }  
29 }  
30 }
```


Hilos en Java

3. Control de vida de los Hilos

¿Qué pasa en un escenario así?

```
1 public class ConcurrentExe {  
2  
3     static class MiHilo extends Thread {  
4         private final int id;  
5  
6         public MiHilo(int id) {  
7             this.id = id;  
8         }  
9  
10        @Override  
11        public void run() {  
12            System.out.println("Hilo " + id + " comenzó su ejecución.");  
13        }  
14    }  
15  
16    Run | Debug | Run main | Debug main  
17    public static void main(String[] args) {  
18        Thread h1 = new MiHilo(id:1);  
19        h1.start();  
20  
21        Thread h2 = new MiHilo(id:2);  
22        h2.start();  
23  
24        System.out.println(x:"El hilo main terminó de lanzar los 2 hilos.");  
25    }  
26 }
```


Hilos en Java

3. Control de vida de los Hilos

```
public class ConcurrentExe {  
  
    static class MiHilo extends Thread {  
        private final int id;  
  
        public MiHilo(int id) {  
            this.id = id;  
        }  
  
        @Override  
        public void run() {  
            System.out.println("Hilo " + id + " comenzó su ejecución.");  
        }  
    }  
  
    public static void main(String[] args) {  
        Thread h1 = new MiHilo(id:1);  
        h1.start();  
  
        Thread h2 = new MiHilo(id:2);  
        h2.start();  
  
        System.out.println(x:"El hilo main terminó de lanzar los 2 hilos.");  
    }  
}
```



Hilos en Java

3. Control de vida de los Hilos

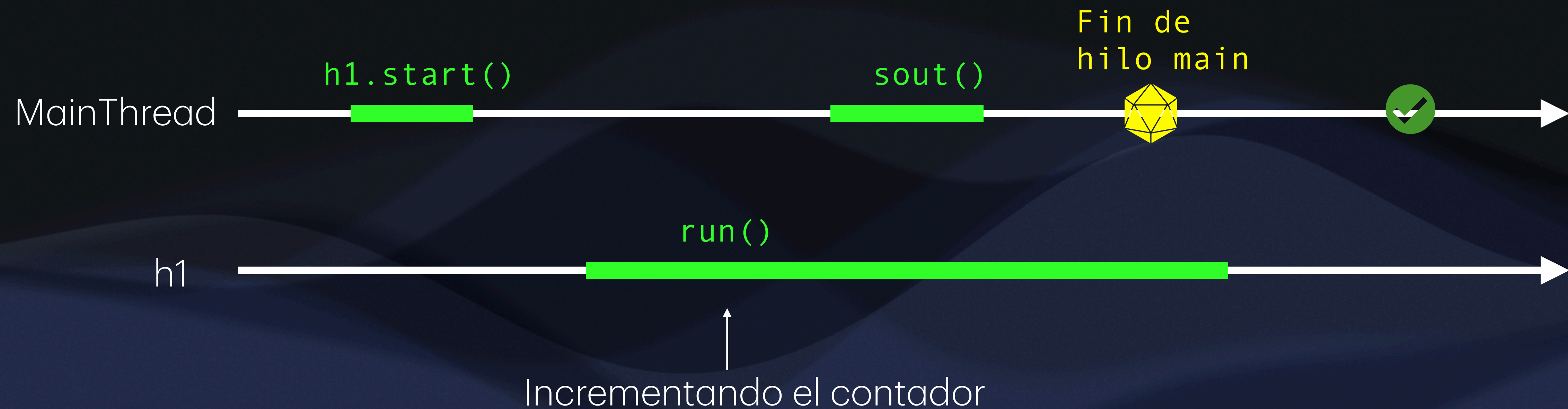
```
1 public class ConcurrentExe {  
2  
3     static class MiHilo  
4         private final  
5  
6  
7  
8
```

```
main terminó de lanzar los 2 hilos.");
```



Hilos en Java

3. Control de vida de los Hilos



Hilos en Java

3. Control de vida de los Hilos

DEL GRIEGO δαίμων QUE
SIGNIFICA ESPÍRITU O GENIO



User Thread	Daemon Thread
JVM wait until user threads to finish their work. It never exit until all user threads finish their work.	The JVM will't wait for daemon threads to finish their work. The JVM will exit as soon as all user threads finish their
JVM will not force to user threads for terminating, so JVM will wait for user threads to terminate	If all user threads have finished their work JVM will force the daemon threads to terminate
User threads are created by the application.	Mostly Daemon threads created by the JVM.
Mainly user threads are designed to do some specific task.	Daemon threads are design as to support the user threads.
User threads are foreground threads.	Daemon threads are background threads.
User threads are high priority threads.	Daemon threads are low priority threads.
Its life independent.	Its life depends on user threads.

Hilos en Java

EL GRIEGO δαίμων QUE
SIGNIFICA ESPÍRITU O GENIO



User Thread

JVM wait until user threads to finish their work. never exit until all user threads finish their work.	
JVM will not force to user threads to finish their work. JVM will wait for user threads to finish their work.	JVM will force the user threads to finish their work.
User threads are created by the user. Daemon threads are created by the JVM.	
Mainly used for long running task. Daemon threads are design as to support the user threads.	
User threads are foreground threads. Daemon threads are background threads.	
User threads are high priority threads. Daemon threads are low priority threads.	
Its life independent. Its life depends on user threads.	

Regresemos al código y veamos qué sucede al descomentar la línea 14

Hilos en Java

3. Control de vida de los Hilos

Método `join()`

El método `join()` permite que un hilo espere a que otro hilo termine su ejecución (`run()`)

```
1 public class JoinThreads {
2     Run | Debug | Run main | Debug main
3     public static void main(String[] args) {
4
5         Thread t2 = new Thread() {
6             @Override
7             public void run() {
8                 System.out.println(x:"T2 está corriendo");
9                 try {
10                     Thread.sleep(millis:1000); // Simula trabajo
11                 } catch (InterruptedException e) {
12                     e.printStackTrace();
13                 }
14                 System.out.println(x:"T2 ha terminado");
15             }
16         };
17
18         Thread t1 = new Thread() {
19             @Override
20             public void run() {
21                 try {
22                     // [Redacted]
23                     System.out.println(x:"T1 continúa después de que T2 terminó");
24                 } catch (InterruptedException e) {
25                     e.printStackTrace();
26                 }
27             }
28         };
29
30         t2.start();
31         t1.start();
32     }
33 }
```



Hilos en Java

3. Control de vida de los Hilos

Método `join()`

El método `join()` permite que un hilo espere a que otro hilo termine su ejecución (`run()`)



```
1 public class JoinThreads {
2     Run | Debug | Run main | Debug main
3     public static void main(String[] args) {
4
5         Thread t2 = new Thread() {
6             @Override
7             public void run() {
8                 System.out.println(x:"T2 está corriendo");
9                 try {
10                     Thread.sleep(millis:1000); // Simula trabajo
11                 } catch (InterruptedException e) {
12                     e.printStackTrace();
13                 }
14                 System.out.println(x:"T2 ha terminado");
15             }
16         };
17
18         Thread t1 = new Thread() {
19             @Override
20             public void run() {
21                 try {
22                     // T1 espera a que T2 termine
23                     t2.join();
24                     System.out.println(x:"T1 continúa después de que T2 terminó");
25                 } catch (InterruptedException e) {
26                     e.printStackTrace();
27                 }
28             }
29         };
30
31         t2.start();
32         t1.start();
33     }
```


Hilos en Java

3. Control de vida de los Hilos

Método `join()`

El método `join()`

a c

Veamos el programa
MainThread

```
1 public class JoinThreads {  
2     Run | Debug | Run main | Debug main  
3     public static void main(String[] args) {  
4  
5         Thread t2 = new Thread() {  
6             @Override  
7             public void run() {  
8                 System.out.println(x:"T2 está corriendo");  
9                 try {  
10                     Thread.sleep(1000); // 1 segundo de trabajo  
11                 } catch (InterruptedException e) {}  
12             }  
13         }  
14         t2.start();  
15  
16         // ...  
17         t2.join();  
18         System.out.println("que T2 terminó");  
19     }  
20 }
```



Hilos en Java

```
public class AlwaysDesignToStop {  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        String threadName = Thread.currentThread().getName();  
        StopCounter counter = new StopCounter();  
        Thread threadA = new Thread(counter);  
        threadA.start();  
        try{  
            Thread.sleep(1500);  
            //threadA.join();  
        }catch(InterruptedException e) {System.out.println(e);}   
        System.out.println("Running Thread: " + threadName);  
        counter.stopcount();  
    }  
}
```

```
class StopCounter implements Runnable{  
    int count=0; boolean done = true;  
  
    int increment() {  
        count++;  
        return count;  
    }  
  
    void stopcount() {  
        this.done = false;  
    }  
  
    @Override  
    public void run() {  
        String threadName = Thread.currentThread().getName();  
        while(this.done) {  
            try{Thread.sleep(500);  
                System.out.println("Running Thread: " + threadName + " increment to: " + increment());  
            }catch(InterruptedException e) {System.out.println(e);}}  
    }  
}
```



Hilos en Java

3. Control de vida de los Hilos

